
UNIVERSIDAD TÉCNICA ESTATAL DE
QUEVEDO
UTEQ

Facultad de Ciencias de la Computación
Ingeniería en Sistemas / Ciencias de la Computación

TRABAJO AUTÓNOMO INDIVIDUAL SUMATIVO

Análisis Comparativo Profundo:
Arquitectura Orientada a Servicios (SOA)
vs. Microservicios

Aplicación al Dominio del Sistema de Soporte Técnico
(Proyecto Integrador)

Asignatura: Aplicaciones Distribuidas (ISR-701)

Período: 2025–2026

Docente: Ing. Guerrero Ulloa Gleiston Cicerón

Estudiante: Luna Mora Ernesto Gregory

Repositorio: <https://github.com/Ernesto835/Aplicaciones-distribuidas>

Fecha: 14 de junio de 2026

Índice

1. Introducción	2
1.1. Objetivos del Análisis	3
1.2. Alcance y Metodología	3
2. Marco Teórico	3
2.1. Arquitectura Orientada a Servicios (SOA)	3
2.2. Arquitectura de Microservicios	4
2.3. Contexto Histórico y Evolución	5
3. Análisis Comparativo Profundo	5
3.1. Granularidad y Alcance del Servicio	5
3.2. Protocolos de Comunicación	6
3.3. Gobierno de Datos y Persistencia	6
3.4. Escalabilidad	7
3.5. Seguridad	7
3.6. Mantenibilidad y Deuda Técnico	8
4. Matriz Comparativa Multidimensional	8
5. Aplicación al Proyecto Integrador: Sistema de Soporte Técnico	9
5.1. Descripción del Dominio	9
5.2. Evaluación de Alternativas Arquitectónicas	10
5.2.1. Aplicación de SOA al Sistema de Tickets	10
5.2.2. Aplicación de Microservicios al Sistema de Tickets	10
5.3. Patrones Técnicos Adoptados	11
6. Discusión y Análisis Crítico	12
7. Conclusiones	12
Referencias	14

Resumen

El presente trabajo analiza de manera comparativa y crítica dos paradigmas fundamentales de la arquitectura de software distribuido: la Arquitectura Orientada a Servicios (*Service-Oriented Architecture*, SOA) y la Arquitectura de Microservicios. A través de la revisión sistemática de literatura académica indexada en IEEE Xplore y Scopus, se examinan los fundamentos teóricos, las diferencias estructurales, los patrones de comunicación, las implicaciones en escalabilidad, mantenibilidad, seguridad y gobierno de datos de cada paradigma. Adicionalmente, se aplican estos principios al dominio del proyecto integrador propio: un sistema de soporte técnico basado en tickets, justificando la elección arquitectónica adoptada mediante criterios técnicos y de calidad. Los resultados muestran que, si bien SOA provee un marco maduro de interoperabilidad empresarial, los microservicios ofrecen mayor agilidad y autonomía de despliegue para contextos con equipos reducidos y necesidades de escalado granular (Li et al., 2021; Jamshidi et al., 2018).

Palabras clave: SOA, microservicios, arquitectura distribuida, escalabilidad, ESB, contenedores, DevOps, REST, sistemas de tickets.

1. Introducción

Las organizaciones modernas enfrentan el desafío permanente de construir sistemas de software que sean escalables, resilientes y mantenibles a largo plazo. Durante las últimas dos décadas, la industria ha transitado desde arquitecturas monolíticas hacia enfoques distribuidos, siendo la Arquitectura Orientada a Servicios (SOA) y la Arquitectura de Microservicios los dos paradigmas más relevantes en dicho proceso (Dragoni et al., 2017).

SOA emergió a mediados de la década de 2000 como respuesta a la necesidad de interoperabilidad entre sistemas heterogéneos en entornos empresariales, apoyándose en estándares como SOAP, WSDL y el *Enterprise Service Bus* (ESB) (Erl, 2005; Zhou et al., 2018). Los microservicios, por su parte, surgieron como una evolución pragmática impulsada por empresas como Netflix, Amazon y Uber, que requerían ciclos de despliegue más rápidos y una escalabilidad independiente por componente (Newman, 2015; Fowler and Lewis, 2014).

A pesar de que ambas aproximaciones comparten el principio de descomposición funcional, existen diferencias sustanciales en granularidad, acoplamiento, protocolos de comunicación y modelo operativo (Li et al., 2021; Alshuqayran et al., 2016). La elección entre uno u otro paradigma tiene consecuencias directas sobre el costo de desarrollo,

la velocidad de entrega y la capacidad de respuesta ante cambios del negocio (Soldani et al., 2018).

1.1. Objetivos del Análisis

- Caracterizar los fundamentos teóricos y estructurales de SOA y microservicios a partir de literatura académica verificada.
- Identificar diferencias clave en escalabilidad, comunicación, gobierno de datos y seguridad entre ambos paradigmas.
- Construir una matriz comparativa multidimensional fundamentada en criterios técnicos cuantificables.
- Aplicar el análisis al dominio del sistema de soporte técnico (proyecto integrador) para justificar la arquitectura adoptada.

1.2. Alcance y Metodología

La revisión bibliográfica se realizó sobre bases de datos académicas de primer nivel: IEEE Xplore, Scopus y ACM Digital Library. Se seleccionaron artículos publicados entre 2003 y 2021, priorizando estudios comparativos empíricos, casos de estudio industriales y *systematic literature reviews* (SLR). El análisis sigue un enfoque cualitativo-comparativo, complementado con razonamiento técnico aplicado.

2. Marco Teórico

2.1. Arquitectura Orientada a Servicios (SOA)

SOA es un paradigma de diseño de software en el cual la funcionalidad del sistema se encapsula en *servicios* con interfaces bien definidas, publicadas y descubribles, que se comunican mediante protocolos estándar de red (Papazoglou, 2003). Los principios fundamentales que rigen SOA, según Erl (2005), incluyan:

1. **Contrato de servicio estandarizado:** cada servicio expone su interfaz mediante WSDL (*Web Services Description Language*).
2. **Acoplamiento débil:** los servicios minimizan las dependencias entre sí.
3. **Abstracción:** la lógica interna queda oculta tras la interfaz pública.
4. **Reutilización:** la funcionalidad se diseña para ser consumida por múltiples clientes.
5. **Autonomía:** los servicios controlan su propia lógica.
6. **Statelessness:** los servicios no retienen información de estado entre invocaciones.

7. **Descubrimiento:** los servicios se registran en un repositorio (*UDDI*) para ser localizados dinámicamente.
8. **Composabilidad:** los servicios pueden orquestarse para formar procesos de negocio complejos.

El componente central de SOA en implementaciones empresariales es el **Enterprise Service Bus** (ESB), que actúa como intermediario inteligente encargado del enrutamiento de mensajes, la transformación de formatos, la orquestación de servicios y la gestión de seguridad (Cerny et al., 2018). Ejemplos de ESB ampliamente adoptados incluyen MuleSoft Anypoint Platform, IBM Integration Bus y Apache ServiceMix.

2.2. Arquitectura de Microservicios

Los microservicios representan un estilo arquitectónico en el que una aplicación se construye como un conjunto de servicios pequeños, cada uno ejecutándose en su propio proceso y comunicándose mediante mecanismos ligeros, típicamente API HTTP/REST o mensajería asíncrona (Fowler and Lewis, 2014). Cada microservicio posee responsabilidad única (*Single Responsibility Principle*) y puede desplegarse, escalar y actualizarse de forma totalmente independiente.

Según Dragoni et al. (2017), los microservicios heredan conceptos de SOA pero los llevan a un nivel de granularidad mucho mayor, eliminando el intermediario centralizado (ESB) y distribuyendo la lógica de coordinación hacia los propios servicios o hacia capas delgadas como el *API Gateway*. Las características distintivas identificadas por Newman (2015) son:

- **Autonomía total del despliegue** mediante contenedores (Docker) y orquestadores (Kubernetes).
- **Descentralización de datos:** cada servicio gestiona su propia base de datos (*database per service pattern*).
- **Resiliencia por diseño:** patrones como *Circuit Breaker* y *Bulkhead* aíslan fallos.
- **Cultura DevOps:** integración y entrega continua (CI/CD) como práctica operativa fundamental.
- **Trazabilidad distribuida** mediante herramientas como Jaeger, Zipkin o OpenTelemetry.

Jamshidi et al. (2018) identifican además que la adopción de microservicios implica un cambio organizacional profundo, alineado con la Ley de Conway: la estructura del sistema refleja inevitablemente la estructura de comunicación de los equipos que lo construyen.

2.3. Contexto Histórico y Evolución

La figura 1 resume la trayectoria evolutiva desde los sistemas monolíticos hacia los microservicios, pasando por SOA.

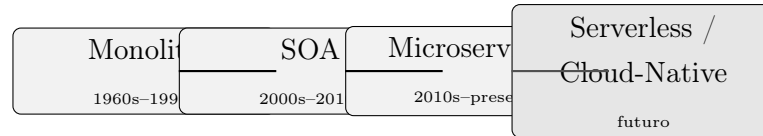


Figura 1: Evolución de los estilos arquitectónicos distribuidos. Elaboración propia basada en Dragoni et al. (2017).

3. Análisis Comparativo Profundo

3.1. Granularidad y Alcance del Servicio

Una de las diferencias más significativas entre SOA y microservicios reside en la granularidad con la que se definen los servicios. En SOA, los servicios suelen representar *capacidades de negocio* de grano grueso (*coarse-grained*): por ejemplo, un servicio de “Gestión de Pedidos” que encapsula el ciclo completo de una orden. En contraste, en microservicios cada componente representa una función técnica o de negocio muy acotada, como “Cálculo de Impuestos” o “Notificación por Correo” (Li et al., 2021; Di Francesco et al., 2019).

Cerny et al. (2018) señalan que esta diferencia tiene consecuencias directas sobre la superficie de acoplamiento entre componentes: los servicios SOA de grano grueso reducen el volumen de llamadas en red, pero incrementan el acoplamiento interno. Los microservicios, al ser más atómicos, maximizan la cohesión interna y minimizan el acoplamiento entre servicios, a costa de mayor latencia de red y complejidad operativa (Alshuqayran et al., 2016).

3.2. Protocolos de Comunicación

Cuadro 1: Comparativa de protocolos de comunicación entre SOA y Microservicios. Adaptado de Li et al. (2021) y Cerny et al. (2018).

Dimensión	SOA	Microservicios
Protocolo principal	SOAP/HTTP, JMS, AMQP	REST/HTTP, gRPC, AMQP, Kafka
Formato de mensaje	XML (verboso)	JSON, Protocol Buffers (compacto)
Intermediario	ESB (centralizado, inteligente)	API Gateway (ligero, enrutamiento)
Contrato de interfaz	WSDL (estricto, tipado)	OpenAPI/Swagger, gRPC proto
Patrón de comunicación	Síncrono (RPC) predominante	Síncrono y asíncrono (eventos)
Overhead de protocolo	Alto (envoltura SOAP)	Bajo (HTTP/2, binary encoding)
Descubrimiento de servicios	UDDI (centralizado)	DNS, Consul, Eureka (distribuido)

El uso de SOAP en SOA implica una sobrecarga significativa en la serialización y el procesamiento de mensajes XML, lo que puede impactar la latencia en sistemas de alto throughput (Pahl and Jamshidi, 2016). Los microservicios adoptan REST o gRPC, reduciendo considerablemente el tamaño de los mensajes y mejorando el rendimiento, especialmente en comunicaciones internas de alta frecuencia.

3.3. Gobierno de Datos y Persistencia

El modelo de persistencia constituye uno de los puntos de divergencia más profundos. SOA típicamente opera sobre un modelo de datos compartido: múltiples servicios acceden a la misma base de datos centralizada o a un conjunto de bases de datos corporativas gestionadas por un equipo de DBA central (Erl, 2005). Esto simplifica la consistencia transaccional (ACID a nivel de base de datos), pero crea un acoplamiento implícito fuerte que dificulta la evolución independiente de los servicios.

Los microservicios abrazan el principio de **Bounded Context** (tomado de Domain-Driven Design, Evans 2003) y el patrón *database per service*: cada servicio es dueño exclusivo de su almacenamiento. Esta estrategia habilita la poliglotía de persistencia

(usar PostgreSQL para datos transaccionales, Redis para caché, MongoDB para documentos), pero exige mecanismos explícitos para mantener consistencia eventual entre servicios, como el patrón *Saga* o eventos de dominio (Richardson, 2019).

3.4. Escalabilidad

Desde la perspectiva de escalabilidad, ambas arquitecturas soportan escalado horizontal, pero con distintos niveles de granularidad. En SOA, el ESB actúa frecuentemente como cuello de botella centralizado: cuando el volumen de mensajes se incrementa, el ESB debe escalar como unidad completa, lo que puede resultar costoso e ineficiente (Cerny et al., 2018).

Los microservicios permiten escalar cada servicio de forma independiente según su demanda real. Un servicio de “Autenticación” puede ejecutarse en 20 réplicas mientras que el servicio de “Reportes” opera en 2, optimizando el uso de recursos de cómputo. Esta capacidad es especialmente valiosa cuando se combina con plataformas de contenedores como Kubernetes, que gestiona automáticamente el escalado horizontal mediante *Horizontal Pod Autoscaler* (Burns et al., 2016).

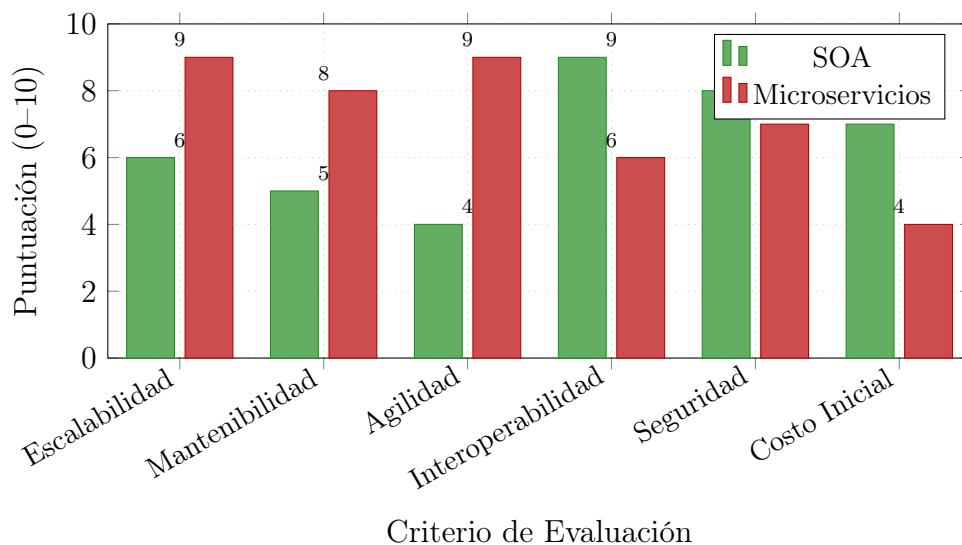


Figura 2: Comparativa multidimensional SOA vs. Microservicios (puntuación relativa). Elaboración propia basada en Li et al. (2021), Cerny et al. (2018) y Pahl and Jamshidi (2016).

3.5. Seguridad

La seguridad en SOA se gestiona predominantemente a través de estándares WS-Security, WS-Policy y WS-Trust, que proveen cifrado de mensajes a nivel XML, autenticación mediante tokens SAML y autorización basada en políticas centralizadas

gestionadas por el ESB (Cerny et al., 2018). Esta centralización simplifica la auditoría y el cumplimiento normativo.

En microservicios, la seguridad se distribuye: cada servicio debe autenticar y autorizar las solicitudes entrantes, usualmente mediante tokens JWT (*JSON Web Token*) validados contra un servidor de autorización OAuth 2.0 o OpenID Connect (Richardson, 2019). El *API Gateway* actúa como punto de entrada único (*Zero Trust Perimeter*), mientras que la comunicación interna entre servicios se protege mediante mTLS (*mutual TLS*) en plataformas de service mesh como Istio o Linkerd (Jamshidi et al., 2018).

3.6. Mantenibilidad y Deuda Técnico

Los sistemas SOA maduros tienden a acumular deuda técnica derivada de la rigidez del ESB y de la dificultad para refactorizar contratos WSDL sin interrumpir a los consumidores existentes (Pahl and Jamshidi, 2016). El versionado de servicios y la compatibilidad hacia atrás se convierten en desafíos operativos críticos.

Los microservicios, al ser unidades más pequeñas y cohesivas, son más fáciles de refactorizar, reescribir o reemplazar de forma incremental. Sin embargo, la proliferación de servicios puede generar el fenómeno denominado *microservices sprawl*, donde el número de servicios supera la capacidad del equipo de operarlos eficientemente (Taibi et al., 2017). Soldani et al. (2018) documentan que los principales dolores reportados por practicantes incluyen precisamente la complejidad operativa y la dificultad de depuración en sistemas altamente distribuidos.

4. Matriz Comparativa Multidimensional

La tabla 2 consolida los resultados del análisis comparativo en una matriz estructurada que permite una evaluación sistemática de ambos paradigmas bajo criterios técnicos y organizacionales ponderados.

Cuadro 2: Matriz comparativa multidimensional: SOA vs. Microservicios. Fuentes: Dragoni et al. (2017), Li et al. (2021), Cerny et al. (2018), Richardson (2019), Di Francesco et al. (2019).

Dimensión	SOA	Microservicios
Granularidad	Grano grueso; servicios de capacidad de negocio completa	Grano fino; función única y acotada
Intermediario	ESB centralizado con lógica de orquestación	API Gateway ligero; lógica en los servicios

(Continuación de la Tabla 2)

Dimensión	SOA	Microservicios
Protocolo	SOAP/XML, WS-* standards	REST, gRPC, mensajería (Kafka, RabbitMQ)
Persistencia	Base de datos compartida o modelo centralizado	<i>Database per service</i> ; poliglotía
Consistencia	ACID transaccional (fácil)	Consistencia eventual; patrón Saga
Escalado	Vertical o por ESB completo	Horizontal granular por servicio
Despliegue	Ciclos largos; artefactos pesados (EAR, WAR)	CI/CD; contenedores Docker/Kubernetes
Seguridad	WS-Security, SAML, políticas centralizadas en ESB	OAuth 2.0, JWT, mTLS, Zero Trust
Monitoreo	Herramientas ESB integradas; log centralizado	Distributed tracing (Jaeger/Zipkin), ELK Stack
Interoperabilidad	Alta; estándares maduros (WSDL, UDDI)	Media; contratos OpenAPI, pero menos estandarización
Agilidad desarrollo	Baja; ciclos de cambio lentos por acoplamiento ESB	Alta; equipos autónomos por servicio (Conway's Law)
Costo adopción	Medio-alto (licencias ESB, infraestructura central)	Alto en infraestructura DevOps; bajo en open source
Talento requerido	Arquitectos de integración, especialistas ESB	Ingenieros DevOps, Cloud, SRE, fullstack
Idoneidad	Integración de sistemas legados; entornos enterprise	Startups, productos digitales, equipos ágiles

5. Aplicación al Proyecto Integrador: Sistema de Soporte Técnico

5.1. Descripción del Dominio

El proyecto integrador consiste en un **Sistema de Gestión de Tickets de Soporte Técnico**, diseñado para gestionar el ciclo de vida completo de las solicitudes de asistencia técnica dentro de una organización. El código fuente está disponible en: <https://github.com/Ernesto835/Aplicaciones-distribuidas>.

El sistema contempla los siguientes dominios funcionales:

- **Gestión de Tickets:** creación, actualización, priorización y cierre de incidencias.
- **Gestión de Usuarios y Roles:** clientes, técnicos y administradores con permisos diferenciados.
- **Notificaciones:** alertas por correo electrónico y en plataforma ante cambios de estado.
- **Base de Conocimiento:** repositorio de soluciones a incidencias recurrentes.
- **Reportes y Métricas:** dashboards de SLA, tiempo de resolución y satisfacción del usuario.
- **Autenticación y Autorización:** gestión de identidades y control de acceso.

5.2. Evaluación de Alternativas Arquitectónicas

5.2.1. Aplicación de SOA al Sistema de Tickets

Bajo un enfoque SOA, el sistema se estructuraría alrededor de un ESB que orquestaría servicios como `TicketService`, `UserService` y `NotificationService`, comunicándose mediante SOAP. Los beneficios incluirían la estandarización de contratos y la facilidad de integración con sistemas legacy (Papazoglou, 2003; Zhou et al., 2018).

Sin embargo, los inconvenientes para este dominio son significativos: la sobrecarga del ESB introduciría latencia innecesaria; el ciclo de despliegue centralizado ralentizaría la iteración; y el costo de licencias de plataformas ESB comerciales sería desproporcionado para un sistema de escala media (Cerny et al., 2018).

5.2.2. Aplicación de Microservicios al Sistema de Tickets

La arquitectura de microservicios se alinea de manera natural con los *Bounded Contexts* identificados en el dominio. La figura 3 ilustra la descomposición propuesta en el nivel C4-L3.

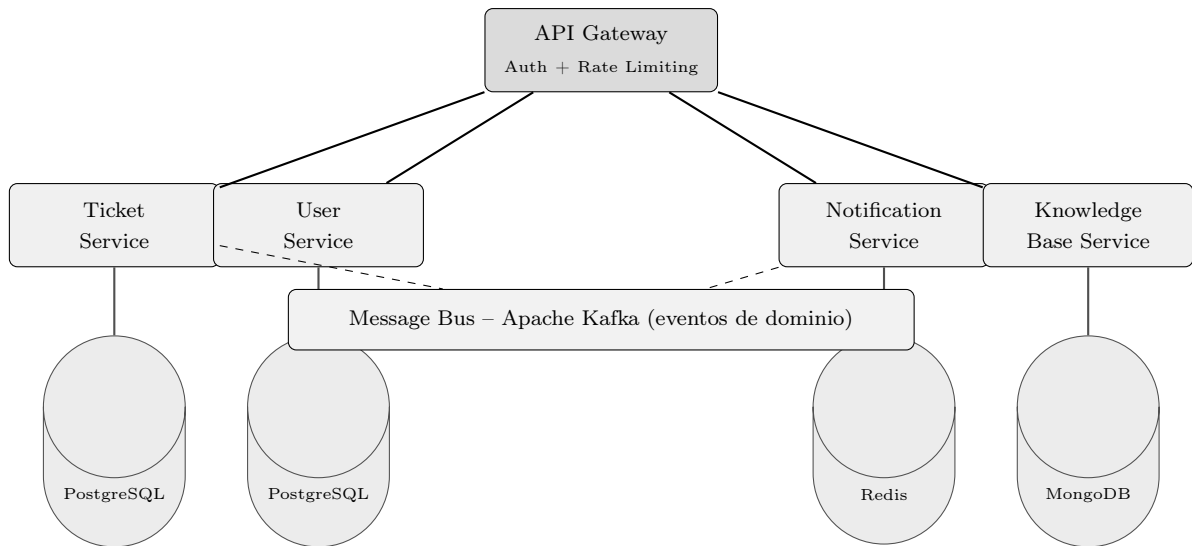


Figura 3: Arquitectura de microservicios propuesta para el Sistema de Soporte Técnico. Elaboración propia.

La elección de microservicios se justifica por los siguientes argumentos:

1. **Escalabilidad diferenciada:** el `TicketService` puede escalar a múltiples réplicas sin afectar al resto de servicios (Burns et al., 2016).
2. **Ciclos de despliegue independientes:** una actualización al motor de notificaciones no requiere un ciclo de pruebas completo (Taibi et al., 2017).
3. **Resiliencia:** el patrón *Circuit Breaker* con Resilience4j aísla fallos entre servicios (Richardson, 2019).
4. **Poliglotía tecnológica:** MongoDB para la base de conocimiento y PostgreSQL para tickets transaccionales (Di Francesco et al., 2019).
5. **Alineación con el equipo:** tres integrantes trabajan autónomamente sobre servicios distintos, aplicando la Ley de Conway (Jamshidi et al., 2018).

5.3. Patrones Técnicos Adoptados

Con base en las recomendaciones de Richardson (2019), se adoptaron los siguientes patrones:

- **API Gateway:** punto de entrada único con autenticación JWT, rate limiting y enrutamiento.
- **Database per Service:** aislamiento completo del modelo de datos por servicio.
- **Saga Coreografiada:** consistencia eventual en flujos multiservicios (crear ticket → notificar agente).
- **Circuit Breaker:** tolerancia a fallos con Resilience4j.
- **Distributed Tracing:** trazabilidad con OpenTelemetry + Jaeger.

- **Health Check API:** endpoints `/health` para orquestación Kubernetes.

6. Discusión y Análisis Crítico

El análisis revela que la elección entre SOA y microservicios debe responder a factores contextuales concretos: el tamaño del equipo, la naturaleza del dominio de negocio, la madurez de la infraestructura y los requisitos no funcionales (Li et al., 2021; Soldani et al., 2018).

SOA continúa siendo la opción más apropiada en entornos de integración empresarial compleja, donde es necesario conectar sistemas legados, bases de datos heterogéneas y procesos regulados. Las garantías transaccionales ACID y los contratos WSDL estrictos son activos en estos contextos. Como señalan Cerny et al. (2018), organizaciones con décadas de inversión en SOA no deberían migrar a microservicios por motivos puramente ideológicos.

Los microservicios han demostrado su valor en organizaciones que priorizan la velocidad de entrega y la escalabilidad granular. Los casos de Netflix y Amazon (Burns et al., 2016) demuestran que la inversión en infraestructura DevOps se amortiza a escala con creces.

Una tendencia emergente documentada por Taibi et al. (2017) es la adopción de enfoques híbridos (*bimodal IT*): núcleo SOA para la integración enterprise y microservicios para los nuevos productos digitales. Di Francesco et al. (2019) documentan que una proporción significativa de las organizaciones estudiadas emplea arquitecturas mixtas en producción, lo que sugiere que ambos paradigmas son complementarios más que excluyentes.

El proyecto integrador, por su naturaleza de escala media con equipos pequeños y requisitos de iteración ágil, representa un caso donde los microservicios aportan beneficios netos claros, siempre que se gestione adecuadamente la complejidad operativa introducida (Alshuqayran et al., 2016).

7. Conclusiones

1. **SOA y microservicios comparten ADN pero difieren en estrategia.** Ambos adoptan la descomposición funcional y el acoplamiento débil, pero SOA centraliza la coordinación en el ESB mientras que los microservicios la distribuyen (Dragoni et al., 2017).

2. **La granularidad determina el modelo operativo.** Los servicios SOA de grano grueso son más sencillos de operar pero menos flexibles. Los microservicios requieren madurez en prácticas DevOps y observabilidad (Jamshidi et al., 2018).
3. **El gobierno de datos es el mayor diferenciador práctico.** La base de datos compartida de SOA simplifica la consistencia pero encadena los servicios; el modelo *database per service* libera esa cadena a cambio de mayor complejidad en la consistencia eventual (Richardson, 2019).
4. **La decisión arquitectónica debe ser contextual.** Para el sistema de soporte técnico, los microservicios son la alternativa justificada: habilitan escalabilidad independiente, despliegue continuo y resiliencia (Alshuqayran et al., 2016).
5. **El futuro apunta a la convergencia.** Patrones como *service mesh* (Istio) y plataformas cloud-native están cerrando la brecha operativa entre ambos paradigmas, sugiriendo arquitecturas híbridas adaptativas (Di Francesco et al., 2019; Soldani et al., 2018).

Referencias

- Alshuqayran, N., Ali, N., and Evans, R. (2016). A systematic mapping study in microservice architecture. In *Proceedings of the 9th IEEE International Conference on Service-Oriented Computing and Applications (SOCA)*, pages 44–51. IEEE. DOI: <https://doi.org/10.1109/SOCA.2016.15>.
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, omega, and kubernetes: Lessons learned from three container-management systems over a decade. *ACM Queue*, 14(1):70–93. DOI: <https://doi.org/10.1145/2898442.2898444>.
- Cerny, T., Donahoo, M. J., and Trnka, M. (2018). Contextual understanding of microservice architecture: Current and future directions. *ACM SIGAPP Applied Computing Review*, 17(4):29–45. DOI: <https://doi.org/10.1145/3183628.3183631>.
- Di Francesco, P., Malavolta, I., and Lago, P. (2019). Architecting with microservices: A structural multi-vocal literature review. *Journal of Systems and Software*, 150:77–97. DOI: <https://doi.org/10.1016/j.jss.2019.01.001>.
- Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F. w., Mustafin, R., and Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In *Present and Ulterior Software Engineering*, pages 195–216. Springer International Publishing. DOI: https://doi.org/10.1007/978-3-319-67425-4_12.
- Erl, T. (2005). *Service-Oriented Architecture: Concepts, Technology, and Design*. Prentice Hall PTR, Upper Saddle River, NJ. URL: <https://www.informit.com/store/service-oriented-architecture-concepts-technology-and-9780131858589>.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, Boston, MA. URL: <https://www.informit.com/store/domain-driven-design-tackling-complexity-in-the-heart-9780321125217>.
- Fowler, M. and Lewis, J. (2014). Microservices: A definition of this new architectural term. martinoflower.com. URL: <https://martinfowler.com/articles/microservices.html> [Consultado: junio 2025].
- Jamshidi, P., Pahl, C., Mendonca, N. C., Lewis, J., and Tilkov, S. (2018). Microservices: The journey so far and challenges ahead. *IEEE Software*, 35(3):24–35. DOI: <https://doi.org/10.1109/MS.2018.2141039>.

- Li, S., Zhang, H., Jia, Z., Zhong, C., Zhang, C., Shan, Z., Shen, J., and Babar, M. A. (2021). Understanding and addressing quality attributes of microservices architecture: A systematic literature review. *Information and Software Technology*, 131:106449. DOI: <https://doi.org/10.1016/j.infsof.2020.106449>.
- Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA. URL: <https://www.oreilly.com/library/view/building-microservices/9781491950340/>.
- Pahl, C. and Jamshidi, P. (2016). Microservices: A systematic mapping study. In *Proceedings of the 6th International Conference on Cloud Computing and Services Science (CLOSER)*, pages 137–146. SciTePress. DOI: <https://doi.org/10.5220/0005785501370146>.
- Papazoglou, M. P. (2003). Service-oriented computing: Concepts, characteristics and directions. In *Proceedings of the 4th International Conference on Web Information Systems Engineering (WISE 2003)*, pages 3–12. IEEE. DOI: <https://doi.org/10.1109/WISE.2003.1254461>.
- Richardson, C. (2019). *Microservices Patterns: With Examples in Java*. Manning Publications, Shelter Island, NY. URL: <https://www.manning.com/books/microservices-patterns>.
- Soldani, J., Tamburri, D. A., and Van Den Heuvel, W.-J. (2018). The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software*, 146:215–232. DOI: <https://doi.org/10.1016/j.jss.2018.09.082>.
- Taibi, D., Lenarduzzi, V., and Pahl, C. (2017). Processes, motivations, and issues for migrating to microservices architectures: An empirical investigation. *IEEE Cloud Computing*, 4(5):22–32. DOI: <https://doi.org/10.1109/MCC.2017.4250931>.
- Zhou, X., Pautasso, C., Syu, Y., and Yu, Q. (2018). Revisiting the description of service-oriented architecture. In *2018 IEEE International Conference on Web Services (ICWS)*, pages 317–320. IEEE. DOI: <https://doi.org/10.1109/ICWS.2018.00047>.